Computer Architecture Project Report

202202984, Division of Computer Engineering, IM CHANGYONG

The "All-rounder Clock" is a standalone, bare-metal digital timekeeper supporting both Stopwatch and Timer functionalities, engineered utilizing x86 Assembly language within the EMU8086 and QEMU environments.

During the initial prototyping phase in the EMU8086 emulator, a critical architectural challenge arose: a conventional polling-based software delay loop caused significant timing errors, as the loop execution speed was heavily dependent on the instruction cycle velocity of the host CPU. To overcome this hardware dependency and resolve timing inconsistencies, the system architecture was systematically refactored to interface directly with the Real-Time Clock (RTC) via BIOS INT 1Ah. By shifting from software-defined polling to hardware-driven state detection, the system achieved deterministic one-second interval tracking.

Finally, the compiled standalone binary was packaged into a standard floppy disk boot image (.img) and deployed using QEMU (Quick Emulator) to evaluate its performance in a near-bare-metal environment. This deployment successfully bypassed emulation overhead, minimized operational jitter, and ultimately delivered high-precision timing accuracy for the comprehensive digital clock system.

1. Introduction & Project Objectives

1.1. Introduction and Project Background

While modern software applications strictly rely on the abstraction layers provided by Operating Systems (OS) such as Windows or Linux, this project focuses on developing a bare-metal, standalone digital clock system that executes immediately upon hardware initialization without any OS dependency. By implementing highly optimized Stopwatch and Timer functionalities directly inside a lightweight bootloader environment, this paper proposes the "All-rounder Clock"—a system designed to provide an intuitive User Interface (UI) while minimizing system resource consumption to near-zero levels.

In low-level systems programming, operating without a kernel requires a comprehensive understanding of computer architecture. This project serves as a practical implementation of a minimalist yet robust system that controls the underlying x86 hardware directly through custom firmware engineering.

1.2. Project Objectives

The primary technical and academic objectives of this project are as follows:

- **Mastery of x86 Real Mode Architecture and BIOS Interrupts:** To achieve a thorough understanding of the x86 register-based control mechanisms using Real Mode BIOS interrupts, specifically focusing on low-level hardware abstraction layers.

- **Direct Hardware Time Management via Real-Time Clock (RTC):** To interface directly with the system's Hardware Real-Time Clock (RTC) using BIOS INT 1Ah or related hardware querying services. This involves retrieving raw time-keeping data

to implement precise, deterministic stopwatch and timer operations.

- **Custom Video Attribute Scheduling and UI Layering:** To design an independent text-mode display interface by manipulating video memory attribute bytes line-by-line. The goal is to achieve an asynchronous, structurally segmented custom UI by dynamically switching background and foreground text color properties to maximize visual ergonomics.

2. Design & Methodology

2.1. Refactoring Timing Control: Shifting from Software Polling to Hardware-Driven RTC Synchronization

The loop-based polling method used in the initial implementation was inextricably bound to the host CPU's clock frequency and instruction cycle velocity. Consequently, a critical limitation emerged where timing errors fluctuated unpredictably depending on the user's specific hardware configuration and specifications. To resolve this architectural discrepancy, the system was refactored to employ a hardware-based Real-Time Clock (RTC) synchronization methodology, drastically improving overall timing precision.

Upon executing the procedure, the system reads the current hardware second parameter formatted as a BCD (Binary Coded Decimal) value and immediately stores it within the BL register. The procedure then continuously polls and monitors the transition of the real-time clock state by performing a continuous validation step (cmp dh, bl) between the dynamic DH register (holding the updating RTC second value) and the static reference state preserved in the BL register. By maintaining a deliberate waiting state if the compared values are identical (je .wait_change), the mechanism successfully captures precise one-second time increments completely independent of the underlying computing execution speeds or host hardware performance specifications.

2.2. Direct Video Attribute Control and Visual UI Layering

To emphasize the characteristics of running directly on bare-metal hardware and to maximize screen readability in text mode, a User Interface (UI) closely resembling the Windows XP Setup screen was created. A bi-level virtual display layout was architecturally designed by utilizing the BIOS video interrupt INT 10h, AH=06h.

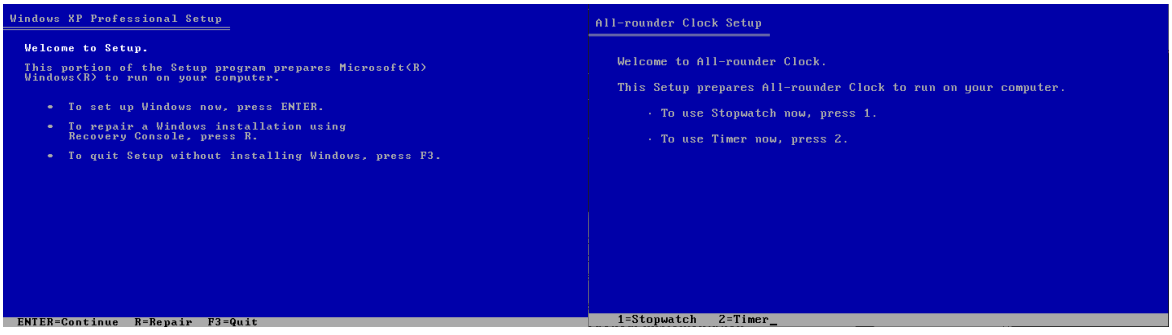

Figure 1. Windows XP Setup(left), All-rounder Clock(right)

When clearing the entire screen, the video attribute byte 17h is mapped into the BH register. This enforces the upper 4 bits of the background color to Blue (1) and the lower 4 bits of the foreground text color to Light Gray (7). To perfectly separate the functional area from the upper screen, the bottom menu guide bar overwrites the region from 1800h (Row 24, Column 0) to 184Fh (Row 24, Column 79) with a separate attribute byte of 70h.

This control method inverts the color matrix to yield Black text (0) on a Light Gray background (7).

2.3. Multi-Sector Boot Loading Strategy

Because the standard x86 Master Boot Record (MBR) restricts the initial boot sector to a strict physical limit of 512 bytes, accommodating the complete stopwatch logic, timer data tracks, and modular UI procedures entirely within Sector 1 is mathematically impossible.

To overcome this architectural barrier, a multi-sector loading strategy was engineered. The primary bootloader initializes the stack at 0x7C00 and immediately triggers BIOS disk I/O via INT 13h, AH=02h to read 15 consecutive sectors from the floppy disk medium, expanding the system boundary by roughly 7.5KB. These sectors, containing the core timekeeping application, are mapped sequentially into the safe memory space starting at address 0x7E00. Once the read operation is verified, a far jump instruction (jmp 0x0000:0x7E00) is executed, effectively transferring hardware control to the clock program's actual entry point

3. Development Environment & Tools Used

To construct, compile, and validate the "All-rounder Clock" inside a standalone bare-metal environment, the following software ecosystem was utilized, shifting from initial prototyping to high-performance simulation:

3.1. EMU8086

Applied during the initial development phase for writing and testing basic code structures. However, due to severe software emulation bottlenecks and throttled execution speeds, it introduced critical latency that distorted real-time rendering. This limitation necessitated a migration to a high-velocity environment.

3.2. Netwide Assembler (NASM)

Adopted to resolve the emulator's processing overhead. It serves as the primary compiler to translate the source code (AllRounder.asm) into a flat, machine-readable format. By using the -f bin output option, it generates a pure binary stream entirely free of OS-specific headers

3.3. Quick Emulator (QEMU)

Utilized as the primary x86 hardware emulator to achieve native-like runtime speeds. Deploying the compiled image (AllRounder.img) via QEMU bypassed application-level virtualization lag. This architectural transition eliminated the processing bottlenecks of EMU8086, restoring perfect timing accuracy and letting the system run seamlessly at true native hardware speeds.

4. Step-by-Step Guide

4.1. Boot Sector

```
[BITS 16]
ORG 0x7C00

; BOOT SECTOR

boot_start:
    xor ax, ax ; better than mov inst
    mov ds, ax
    mov es, ax
    mov ss, ax
    mov sp, 0x7C00

    mov ah, 02h
    mov al, 15
    mov ch, 0
    mov cl, 2
    mov dh, 0
    mov bx, 0x7E00
    int 13h
    jc boot_start

    jmp 0x0000:0x7E00

times 510-($-$$) db 0
dw 0xAA55
```

The code [BITS 16] explicitly instructs the NASM assembler to compile the subsequent instructions into 16-bit Real Mode machine code. After that, the ORG directive informs the assembler that the program will be loaded and executed at the memory address 0x7C00. This sets the baseline for all variables and jump addresses within the code. The AX register is cleared to 0 by performing an XOR operation on itself. The value of AX is then assigned to the DS, ES, and SS registers to unify the baseline of the segment registers to 0x0000. The SP register is set to 0x7C00, which is the starting address of the bootloader. Since the stack in the x86 architecture decrements and grows downward as data is pushed, the empty memory area below 0x7C00 is secured as the stack space.

To select the disk sector reading function among INT 13h services, the function number 02h is placed into the AH register. The number of sectors to read is specified in the AL register. This is because the boot sector space is insufficient, so the subsequent "All-rounder Clock" main program part needs to be brought in large quantities at once. The disk track number to read is specified in the CH register, and the starting sector number 2 is specified in the CL register. Since sector 1 is the running bootloader itself, this means reading will start from the immediate next sector. The disk head number to read is specified in the DH register, and the buffer address to store the data read from the disk is specified in the BX register. Since the ES register was initialized to 0, the clock program main body is sequentially copied to the address range 0x0000:0x7E00. Based on the configured register values, the BIOS disk I/O interrupt is called by the int 13h instruction to read data from the actual floppy disk interface. If an error occurs during the disk read operation, the CPU Carry Flag becomes 1; to prevent data loss, the execution returns to the first point of the read routine (boot_start) and retries until the Carry Flag becomes 0.

This is the padding code to meet the physical size rules of the Master Boot Record (MBR). Excluding the code size filled from the current position ($) to the boot sector starting point ($$), all remaining space is completely filled with 0 (db 0) to make it exactly 510 bytes in size. The last 2 bytes of the boot sector are finished with the magic number 0xAA55. The PC BIOS checks whether this signature value is accurately embedded at the end of the first sector of the disk, determines that it is a "normal bootable medium," and hands over control. In this way, the boot sector becomes exactly 512 bytes.

4.2. Data Area

```
clock_program_start:
    jmp main_menu

; DATA

S_HOUR            dw 0
S_MINUTE          dw 0
S_SECOND          dw 0
T_HOUR          dw 0
T_MINUTE        dw 0
T_SECOND        dw 0
LAP_COUNT       dw 0
LAP_HOUR        times 10 dw 0
LAP_MIN         times 10 dw 0
LAP_SEC         times 10 dw 0

menu_title        db 'All-rounder Clock Setup', 0
title_underbar   db 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 0
menu_line1        db 'Welcome to All-rounder Clock.', 0
menu_line2        db 'This Setup prepares All-rounder Clock to run on your computer.', 0
menu_line3        db 249, ' To use Stopwatch now, press 1.', 0
menu_line4        db 249, ' To use Timer now, press 2.', 0
menu_key          db '1=Stopwatch    2=Timer', 0
stopwatch_text   db 'Stopwatch                    ', 0
stopwatch_key    db 'P=Pause    L=Lap    R=Reset    F3=Quit', 0
stopwatch_pause db 'S=Start', 0
timer_text         db 'Timer                          ', 0
timer_key          db 'P=Pause    F3=Quit', 0
timer_pause        db 'S=Start', 0
timeout_key        db 'F3=Exit              ', 0
input_hour_msg   db 'Enter Hour    (00-23): ', 0
input_min_msg    db 'Enter Minute (00-59): ', 0
input_sec_msg    db 'Enter Second (00-59): ', 0
timeout_msg       db 'Times Out!', 0
```

The **DATA area** is the designated section where all variables and strings required for the operation of the "All-rounder Clock" program are organized in memory. The clock_program_start label jumps directly to the actual main screen rendering label (main_menu), bypassing the DATA area to prevent the processor from misinterpreting raw data bytes as executable code instructions.

The **dw (Define Word) region** manages variables that require numerical operations. The variables tracking the current stopwatch time are designated as HOUR, MINUTE, and SECOND, while the countdown timer variables are explicitly prefixed with T_ (e.g., T_HOUR, T_MINUTE, T_SECOND). To implement the stopwatch lap-time logging feature, tracking variables are prefixed with LAP_, and a dedicated array space is secured using the times 10 macro to allow a maximum of 10 sequential records.

The **db (Define Byte) region** handles variables necessary for text string output. Alphanumeric structures such as the welcoming message and the main menu navigation guides are declared as byte-unit strings. Every string definition is concluded with a NULL terminator (0) at the absolute trailing boundary, enabling the low-level string printing loop to accurately detect the end of the data stream. Furthermore, the numerical constants 205 and 249 are explicitly embedded to represent the structural horizontal double-line (═) and the menu bullet point (·) characters in text mode, reconstructing the visual layout of the legacy interface.

4.3. Main Menu Area

```
; MAIN MENU

main_menu:
    call clear_screen
    mov si, menu_title
    call draw_header
    mov dh, 4
    mov dl, 4
    call gotoxy
    mov si, menu_line1
    call print_string
    mov bl, 17h
    mov dh, 6
    mov dl, 4
    call gotoxy
    mov si, menu_line2
    call print_string
    mov dh, 8
    mov dl, 8
    call gotoxy
    mov si, menu_line3
    call print_string
    mov dh, 10
    mov dl, 8
    call gotoxy
    mov si, menu_line4
    call print_string
    mov si, menu_key
    call draw_statusbar

wait_menu:
    mov ax, 0100h
    int 16h
    jz wait_menu
    mov ah, 00h
    int 16h
    cmp al, '1'
    je sw_main
    cmp al, '2'
    je timer_main
    jmp wait_menu

go_main:
    jmp main_menu
```

The main_menu label establishes the primary user interface layout by leveraging absolute coordinate positioning. The structural design operates entirely by manipulating the cursor offset location through the gotoxy procedure before rendering each text block. By explicitly mapping the destination coordinates—such as Row 4, Column 4 (dh=4, dl=4) for the initial welcoming line—the system systematically structures the menu interface line-by-line, sequentially invoking print_string to print the hardcoded data string constants preserved in memory. The interface concludes by calling draw_statusbar to mount the navigation guide string (menu_key) on the isolated bottom row.

Once the screen composition is complete, the program enters the wait_menu label to monitor the user's keyboard input via the INT 16h instruction. By executing mov ax, 0100h, it checks whether a new key has entered the keyboard buffer. If no key has been pressed, the Zero Flag becomes 1, and the system repeats this operation infinitely due to the jz condition until a keyboard input is received. When user input is detected (ZF=0), the system immediately reads the ASCII value of the entered key into the AL register by executing mov ah, 00h. The retrieved value undergoes comparison operations; if the value matches '1' or '2', the je instruction is triggered, transferring system control to the stopwatch_main and timer_main labels, respectively. If any other key is pressed, it is ignored, and the program returns to wait_menu to repeat the operation.

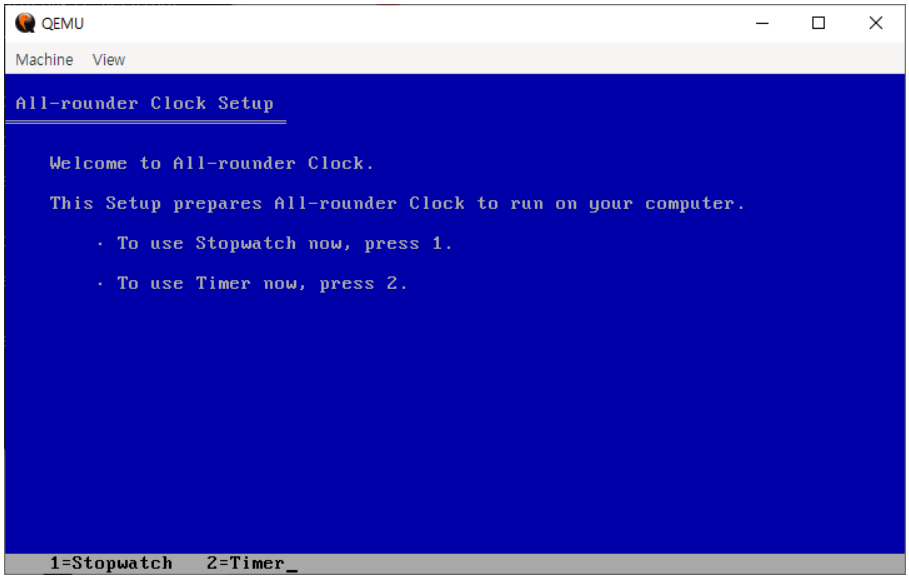

Figure 2. Main Menu UI

4.4. Stopwatch Area

```
; STOPWATCH

sw_main:
    call clear_screen
    mov word [S_HOUR], 0
    mov word [S_MINUTE], 0
    mov word [S_SECOND], 0
    mov word [LAP_COUNT], 0
    mov si, stopwatch_text
    call draw_header
    mov si, stopwatch_key
    call draw_statusbar

sw_display:
    mov bl, 17h
    mov dh, 5
    mov dl, 4
    call gotoxy
    mov ax, [S_HOUR]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [S_MINUTE]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [S_SECOND]
    call print_two_digit

    call delay
    mov ah, 01h
    int 16h
    jz sw_inc
    mov ah, 00h
    int 16h
    or al, 20h
    cmp al, 'p'
    je sw_pause
    cmp al, 'l'
    je sw_lap
    cmp al, 'r'
    je sw_main
    cmp ah, 3Dh
    je go_main
    jmp sw_inc

sw_pause:
    mov si, stopwatch_pause
    call draw_statusbar

sw_wait:
    mov ah, 01h
    int 16h
    jz sw_wait
    mov ah, 00h
    int 16h
    or al, 20h
    cmp al, 's'
    je sw_resume
    cmp al, 'l'
    je sw_lap_paused
    cmp al, 'r'
    je sw_main
    cmp ah, 3Dh
    je go_main
    jmp sw_wait
```

The stopwatch_main label invokes the clear_screen procedure to erase all existing display content, and then initializes the stopwatch-related time variables and the LAP_COUNT variable to 0. It prints the stopwatch title in the upper header region and renders the corresponding operational control keys on the bottom status bar.

The sw_display label retrieves the current stopwatch time values and prints them in real-time in the HH:MM:SS format at Row 5, Column 4 (mov dh, 5, mov dl, 4) using the print_two_digit and print_char procedures. Afterward, it calls the hardware-RTC synchronization routine, the delay label, to generate an exact one-second time delay. Once the delay concludes, the system immediately checks the keyboard buffer via the INT 16h, AH=01h interrupt. If no user input is confirmed, the Zero Flag is set to 1, causing the program to branch to the sw_inc label, which is the routine that increments the time.

When a keyboard input is detected (ZF=0), the system extracts the corresponding key from the buffer using the AH=00h interrupt. By executing the or al, 20h operation, the entered alphanumeric character is converted to lowercase, effectively eliminating case sensitivity. The system then jumps to the respective assigned labels based on the input; if the F3 key (scan code 3Dh) is pressed, it escapes to the go_main label. Any other irrelevant key inputs are ignored, and the system branches normally to the sw_inc label to continue incrementing the time.

The sw_pause label, which is triggered when the user presses the 'p' key, transitions the system into a paused state. It updates the bottom guide bar by replacing the P=Pause hint with S=Start, and then monitors for subsequent keyboard inputs within the sw_wait loop. Because this loop does not invoke the delay or sw_inc labels, the stopwatch time is effectively frozen. Instead, due to the jz sw_wait condition, the processor continuously polls only the keyboard buffer status. From this state, if the 's', 'l', 'r', or F3 keys are detected, system control is cleanly transferred to their respective mapped labels.

```
sw_resume:
    mov si, stopwatch_key
    call draw_statusbar
    jmp sw_display

sw_lap:
    call do_lap
    jmp sw_inc

sw_lap_paused:
    call do_lap
    jmp sw_wait

sw_inc:
    add word [S_SECOND], 1
    cmp word [S_SECOND], 60
    jl sw_display
    mov word [S_SECOND], 0
    add word [S_MINUTE], 1
    cmp word [S_MINUTE], 60
    jl sw_display
    mov word [S_MINUTE], 0
    add word [S_HOUR], 1
    cmp word [S_HOUR], 24
    jl sw_display
    mov word [S_HOUR], 0
    jmp sw_display
```

When the user enters the 's' key in the sw_wait label, the program transitions to the sw_resume label. At this stage, it executes the process of changing the S=Start hint back to P=Pause and moves to the sw_display label to resume the stopwatch time increment. The sw_lap and sw_lap_paused labels manage the lap-time logging mechanism when the user presses the 'l' key. Both labels invoke the internal lap-logging label, do_lap; afterward, if the stopwatch was actively running, the program jumps to the sw_inc label to maintain sequential time increments, whereas if it was in a paused state, it branches back to sw_wait to sustain the standby condition.

The sw_inc label is a core component for implementing the stopwatch time, executed after the one-second delay concludes to sequentially update the clock variables through a 60-base (minute, second) and 24-base (hour) carry-over logical computing structure:

- It first increments the second register (S_SECOND) by 1. If this value is strictly less than 60 (jl sw_display), it bypasses further arithmetic modifications and returns directly to the display routine to update the screen with the modified second value.
- Once the second value reaches 60, it resets S_SECOND to 0 and adds 1 to the minute register (S_MINUTE). If the updated minute value remains less than 60, the program branches immediately to the screen display routine without a carry-over.
- When the minute value hits 60, it clears S_MINUTE to 0 and increments the hour register (S_HOUR) by 1. If the hour register is less than 24, it transitions directly to the display routine; however, if it reaches 24, indicating the passage of a full day, the program recycles S_HOUR back to 0 before branching to sw_display.

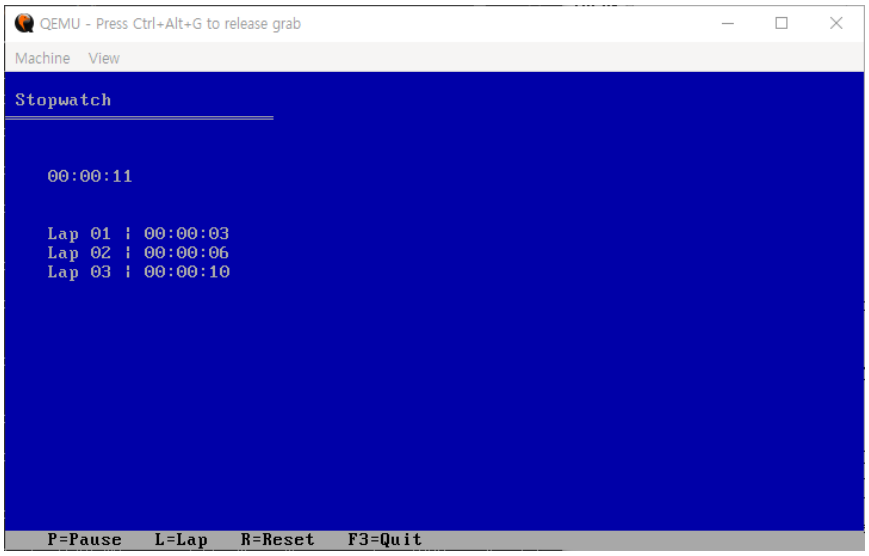

Figure 3. Stopwatch Section UI

4.5. Timer Area

```
; TIMER

timer_main:
    call clear_screen
    mov si, timer_text
    call draw_header
    mov dh, 5
    mov dl, 4
    call gotoxy
    mov si, input_hour_msg
    call print_string
    call bios_input_two_digit
    mov [T_HOUR], ax
    mov dh, 7
    mov dl, 4
    call gotoxy
    mov si, input_min_msg
    call print_string
    call bios_input_two_digit
    mov [T_MINUTE], ax
    mov dh, 9
    mov dl, 4
    call gotoxy
    mov si, input_sec_msg
    call print_string
    call bios_input_two_digit
    mov [T_SECOND], ax
    call clear_screen
    mov si, timer_text
    call draw_header
    mov si, timer_key
    call draw_statusbar

timer_display:
    mov bl, 17h
    mov dh, 5
    mov dl, 4
    call gotoxy
    mov ax, [T_HOUR]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [T_MINUTE]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [T_SECOND]
    call print_two_digit

    mov ax, [T_HOUR]
    or   ax, [T_MINUTE]
    or   ax, [T_SECOND]
    jz   timer_timeout

    call delay
    mov ah, 01h
    int 16h
    jz timer_dec
    mov ah, 00h
    int 16h
    or al, 20h
    cmp al, 'p'
    je timer_pause_fn
    cmp ah, 3Dh
    je go_main
    jmp timer_dec
```

The timer_main label clears the screen when the timer mode is executed and draws the upper header. After that, it sequentially moves the cursor to Row 5 Column 4, Row 7 Column 4, and Row 9 Column 4, and prints a message (input_msg) requesting the user to input hours, minutes, and seconds. Immediately after printing the message, it calls the call bios_input_two_digit label to receive a two-digit number from the keyboard. After the input is completed, the values returned to the AX register are stored (mov) into the memory addresses of the timer configuration variables T_HOUR, T_MINUTE, and T_SECOND, respectively. When the time setting is finished, it completely clears the screen again and places the timer-specific key guide status bar (timer_key).

The timer_display label reads the T_HOUR, T_MINUTE, and T_SECOND values remaining in the current memory and renders them in real-time at the position of Row 5 Column 4 in the HH:MM:SS format. Immediately after printing the time, the three variables (T_HOUR, T_MINUTE, T_SECOND) are combined and inspected through or operations. If all three values become 0 at the same time, the result of the AX register becomes 0, so the Zero Flag is set (ZF=1), which satisfies the jz condition, and the program immediately jumps to the timer_timeout label, which is the timer expiration notification label.

If there is still time left on the timer, it gives a 1-second delay with call delay, which is the hardware RTC synchronization routine, and then checks the keyboard with the INT 16h, AH=01h interrupt. If no key is pressed, it moves to the timer_dec label, which is the routine where time decreases. If a key is inputted, it is pulled out with AH=00h and changed to lowercase; if it is 'p', it escapes to the pause routine (timer_pause_fn), and if it is the F3 scan code (3Dh), it escapes to the main menu (go_main), and other keys are ignored and it proceeds normally to the timer_dec label where time decreases.

```
timer_pause_fn:
    mov si, timer_pause
    call draw_statusbar
timer_wait_resume:
    mov ah, 01h
    int 16h
    jz timer_wait_resume
    mov ah, 00h
    int 16h
    or al, 20h
    cmp al, 's'
    je timer_resume
    cmp ah, 3Dh
    je go_main
    jmp timer_wait_resume
timer_resume:
    mov si, timer_key
    call draw_statusbar
    jmp timer_display

timer_dec:
    cmp word [T_SECOND], 0
    jg dec_sec
    cmp word [T_MINUTE], 0
    jg borrow_min
    cmp word [T_HOUR], 0
    je timer_display
    dec word [T_HOUR]
    mov word [T_MINUTE], 59
    mov word [T_SECOND], 59
    jmp timer_display
borrow_min:
    dec word [T_MINUTE]
    mov word [T_SECOND], 59
    jmp timer_display
dec_sec:
    dec word [T_SECOND]
    jmp timer_display

timer_timeout:
    call clear_screen
    mov bl, 17h
    mov dh, 12
    mov dl, 35
    call gotoxy
    mov si, timeout_msg
    call print_string
    mov si, timeout_key
    call draw_statusbar
wait_timeout:
    mov ah, 01h
    int 16h
    jz wait_timeout
    mov ah, 00h
    int 16h
    cmp ah, 3Dh
    je go_main
    jmp wait_timeout
```

The timer_pause_fn label replaces the bottom status bar guide with timer_pause (S=Start) upon entering the paused state and then monitors for keyboard input again in the timer_wait_resume loop. This loop maintains the paused state of the timer, and the processor continuously checks only the keyboard buffer status. From this state, if the 's' key is inputted, the program moves to the timer_resume label, restores the status bar back to timer_key (P=Pause), and jumps to the timer_display label to resume the countdown. If the F3 scan code (3Dh) is detected, it escapes to the main menu through the go_main label.

The timer_dec label manages the 60-base and 24-base borrow (Borrow) operation structures, sequentially decrementing the timer variables at each 1-second interval:

- It first inspects the second variable (T_SECOND) with the cmp instruction. If the value is greater than 0 (jg dec_sec), it branches to the dec_sec label, decreases T_SECOND by 1 via the dec instruction, and returns to timer_display.
- If T_SECOND is 0, it checks the minute variable (T_MINUTE). If T_MINUTE is greater than 0 (jg borrow_min), it branches to the borrow_min label, decreases T_MINUTE by 1, resets T_SECOND to 59, and returns to timer_display.
- If both seconds and minutes are 0, it checks the hour variable (T_HOUR). If even T_HOUR is 0 (je timer_display), it skips the subtraction operation. If T_HOUR is greater than 0, it decreases T_HOUR by 1, fills both T_MINUTE and T_SECOND with 59 to perform the borrow operation, and returns to timer_display.

The timer_timeout label is executed when the countdown reaches 00:00:00. It clears the screen through the clear_screen label, moves the cursor, and prints the expiration notification message in the center of the screen. Afterward, it places the timeout_key guide on the bottom status bar and enters the wait_timeout loop. This loop continuously polls the keyboard buffer using the INT 16h interrupt, completely ignores all other key inputs, and repeats infinitely until the F3 scan code (3Dh) is pressed, and when the key is pressed, it triggers the je go_main instruction to safely return the user to the main menu.

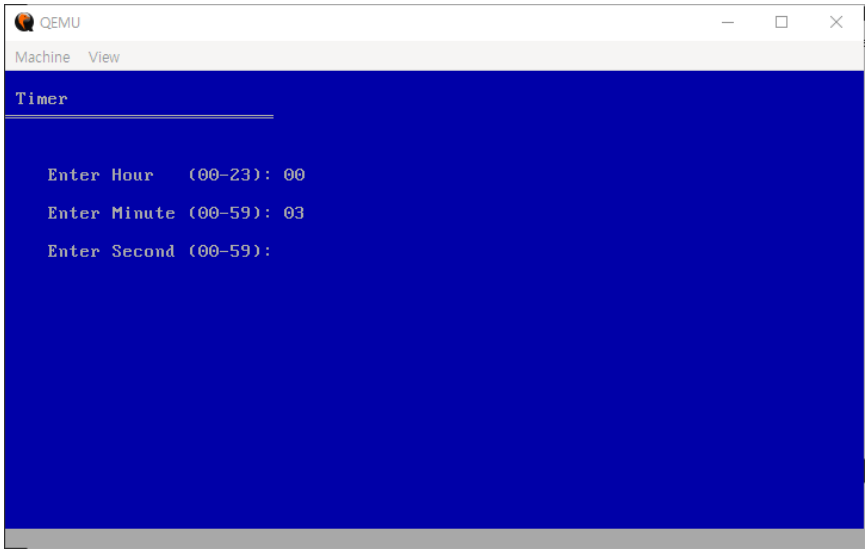

Figure 4. Timer Setting Section UI

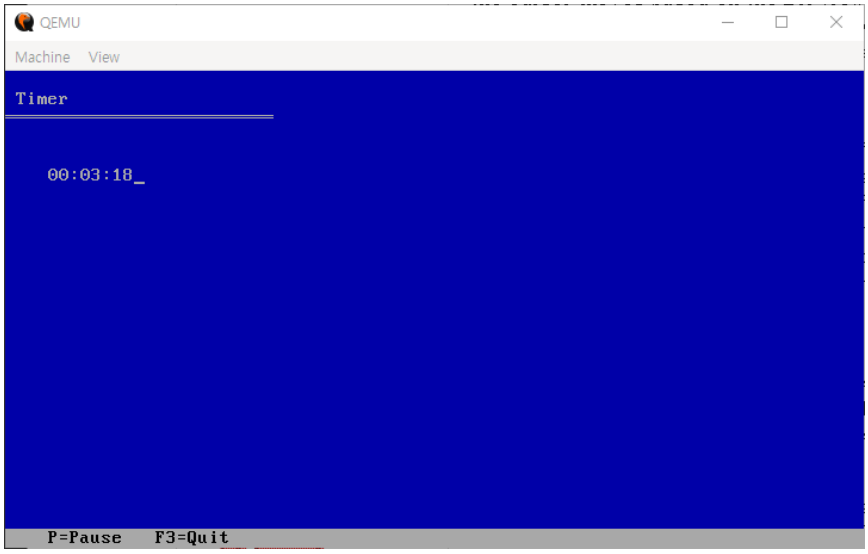

Figure 5. Timer Section UI

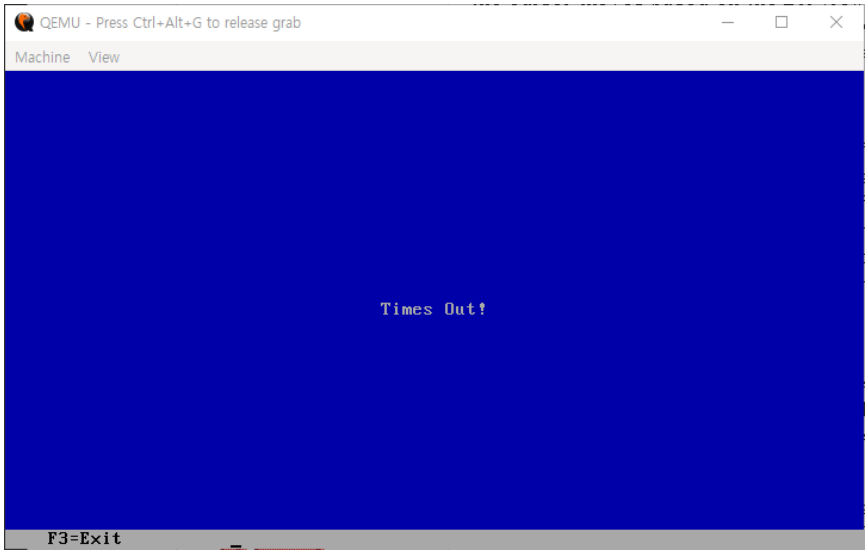

Figure 6. Timeout Section UI

4.6. Procedures Area

```
gotoxy:
    push ax
    push bx
    mov ah, 02h
    mov bh, 0
    int 10h
    pop bx
    pop ax
    ret

draw_header:
    mov bl, 17h
    mov dh, 1
    mov dl, 1
    call gotoxy
    call print_string
    mov dh, 2
    mov dl, 0
    call gotoxy
    mov si, title_underbar
    call print_string
    ret

draw_statusbar:
    mov bl, 70h
    mov dh, 24
    mov dl, 4
    call gotoxy
    call print_string
    ret

print_string:
    push ax
    push si
.print_loop:
    lodsb
    or al, al
    jz .print_done
    call print_char
    jmp .print_loop
.print_done:
    pop si
    pop ax
    ret

print_char:
    push ax
    push bx
    mov ah, 0Eh
    mov bh, 0
    int 10h
    pop bx
    pop ax
    ret
```

The gotoxy label is a function that moves the screen's cursor position to specific coordinates. At the start of the function, to prevent corruption of the existing register values currently in use, it backs up the values onto the stack using the push ax and push bx instructions. Afterward, it invokes the INT 10h, AH=02h (BIOS Video Services - Set Cursor Position) interrupt. At this time, the screen page number BH is designated as 0, and the cursor moves based on the DH (row) and DL (column) coordinates set by the caller before invocation. When the operation is complete, it restores the backed-up register values with the pop instruction and returns via ret.

The draw_header label plays the role of drawing the title and an underbar at the top of the screen. After specifying the color attribute matrix value (BL=17h, light gray text on a blue background), it moves to the position of Row 1 Column 1 and outputs the original title string. Right after that, it moves the cursor to Row 2 Column 0 and outputs the double-line character array title_underbar to render a boundary line at the bottom of the title.

The draw_statusbar label outputs the key guide in the bottommost status bar region. After specifying the color attribute as BL=70h (black text on a light gray background), it moves the cursor to the bottommost position of Row 24 Column 4 and renders the received guide string.

The print_string label is a function that sequentially outputs a string stored in memory to the screen. Through the lodsb instruction, it reads 1-byte character data from the address pointed to by the SI register, loads it into the AL register, and automatically increments the SI address by 1 at the same time. Afterward, through the or al, al operation, it inspects whether the retrieved character is a NULL character (0) that signals the end of the string. If it is 0, the Zero Flag is set (ZF=1), terminating the output due to the jz .print_done condition; if a character remains, it calls print_char to print the character and then returns to jmp .print_loop to repeat this infinitely.

The print_char label is a minimum-unit function that outputs a single character contained in the AL register to the screen. To prevent register corruption, it backs up AX and BX onto the stack, and then invokes the INT 10h, AH=0Eh (BIOS Video Services - Teletype Output) interrupt to render the character at the current cursor position on the screen and automatically moves the cursor one space to the right. When the operation is finished, it restores the backed-up registers.

```
print_two_digit:
    push ax
    push bx
    push cx
    mov cl, bl
    xor ah, ah
    mov bl, 10
    div bl
    mov bl, cl
    add al, '0'
    call print_char
    mov al, ah
    add al, '0'
    call print_char
    pop cx
    pop bx
    pop ax
    ret

bios_input_two_digit:
    push bx
    push cx
    mov bl, 17h
.w1:
    mov ah, 00h
    int 16h
    cmp al, 13
    je .sd
    cmp al, '0'
    jl .w1
    cmp al, '9'
    jg .w1
    call print_char
    sub al, '0'
    mov cl, al
.w2:
    mov ah, 00h
    int 16h
    cmp al, 13
    je .j1
    cmp al, '0'
    jl .w2
    cmp al, '9'
    jg .w2
    call print_char
    sub al, '0'
    mov ch, al
    mov al, cl
    mov bl, 10
    mul bl
    add al, ch
    xor ah, ah
    jmp .ex
.sd: xor ax, ax
    jmp .ex
.j1: mov al, cl
    xor ah, ah
.ex:
    pop cx
    pop bx
    ret
```

The print_two_digit label is a function that outputs a 1-byte integer value to the screen in a 2-digit decimal format (e.g., 05, 12). To prevent corruption of the values currently in use before calling the function, it backs up the AX, BX, and CX registers onto the stack. To prevent the BL value, which contains the text color attribute, from being destroyed during the division operation, it is temporarily copied to the CL register.

To separate the number contained in AX into decimals, the upper byte is cleared (xor ah, ah), BL is set to 10, and the division instruction div bl is performed. As a result of the operation, the quotient (tens digit) is stored in AL, and the remainder (ones digit) is stored in AH. When the division is finished, the evacuated color attribute is restored to its original state (mov bl, cl). The ASCII offset value '0' is added to the quotient contained in AL to convert it into a character, and then the tens digit is outputted first via print_char. Subsequently, the remainder that was contained in AH is moved to AL, converted into a character in the same manner, and the ones digit is outputted. When all operations are completed, the backed-up registers are restored and the function returns.

The bios_input_two_digit label is a function that receives keyboard input from the user when configuring the timer and synthesizes a 2-digit integer value. During input, the real-time background rendering color is set to light gray text on a blue background (BL=17h).

The w1 label, which handles the first digit input, infinitely waits for keyboard input via the INT 16h, AH=00h interrupt. If the user presses the Enter key (ASCII 13) immediately without entering a number, it branches to the .sd label and processes the resulting value as 0 (xor ax, ax). If an irrelevant key other than a number key is pressed, it is ignored and the standby loop repeats. When a valid number is inputted, it prints the character on the screen, converts the ASCII value into an integer (sub al, '0'), and stores it in the CL register, which is the temporary register for the tens digit.

The w2 label, which handles the second digit input, waits for input again through the interrupt. If the Enter key (ASCII 13) is pressed while the first number has been inputted, it moves to the j1 label, treats the first input value (CL) directly as the ones digit value, clears the upper byte, and terminates. In the same manner, keys other than numbers are filtered out, and when a valid number is inputted, it prints the character on the screen, converts it into an integer (sub al, '0'), and stores it in the CH register, which is the temporary register for the ones digit.

Integer Value Synthesis and Return: When both digits are successfully inputted, the CL value, which was the tens digit, is moved into AL, and then multiplied by 10 (mul bl with BL=10). By adding the CH value, which is the ones digit, to this resulting value, the final 2-digit integer value is completed. To satisfy the return specification, the AH register is cleanly cleared (xor ah, ah), and the backed-up registers are restored before returning to the caller.

```
do_lap:
    mov ax, [LAP_COUNT]
    cmp ax, 10
    jge .done
    mov bx, ax
    shl bx, 1
    mov ax, [S_HOUR]
    mov [LAP_HOUR + bx], ax
    mov ax, [S_MINUTE]
    mov [LAP_MIN + bx], ax
    mov ax, [S_SECOND]
    mov [LAP_SEC + bx], ax
    inc word [LAP_COUNT]
    call print_laps
.done:
    ret

print_laps:
    push ax
    push bx
    push cx
    push dx
    push si
    mov cx, [LAP_COUNT]
    jcxz .ldone
    mov bx, 0
.l_loop:
    push bx
    mov bl, 17h
    mov dh, [esp]
    add dh, 8
    mov dl, 4
    call gotoxy
    mov si, .lap_str
    call print_string
    mov ax, [esp]
    inc ax
    call print_two_digit
    mov al, ' '
    call print_char
    mov al, '|'
    call print_char
    mov al, ' '
    call print_char
    mov ax, [esp]
    shl ax, 1
    mov si, ax
    mov ax, [LAP_HOUR + si]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [LAP_MIN + si]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [LAP_SEC + si]
    call print_two_digit
    pop bx
    inc bx
    loop .l_loop
.ldone:
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
.lap_str db 'Lap ', 0
```

The do_lap label is a function that sequentially records the current stopwatch time data (hours, minutes, seconds) into the array memory every time the user presses the lap key. First, it reads the LAP_COUNT value, which is the number of lap times recorded up to the current moment, into the AX register and compares it with the maximum storage limit of 10 (cmp ax, 10). If the value is 10 or greater, it does not record any further to prevent array buffer overflow and jumps to the .done label to terminate. If it is less than 10, it copies the count value to BX and then performs the bitwise left shift instruction shl bx, 1. Since we previously declared the lap time array as a 16-bit word size (dw) in the data segment, this is a precise low-level address control operation that multiplies the index by 2 to align with the physical byte offset address unit. Once the index alignment is complete, it reads the current stopwatch time values, S_HOUR, S_MINUTE, and S_SECOND, respectively, and sequentially stores them into the calculated memory offset addresses [LAP_HOUR + bx], [LAP_MIN + bx], and [LAP_SEC + bx]. When the recording is completed, it increments the LAP_COUNT variable by 1 (inc), calls print_laps to update the list in real-time, and then returns.

The print_laps label is a function that sequentially lists and outputs all lap time data cumulatively stored in the array memory to the screen. To prevent corruption of the register values currently in use during the execution of the function, it backs up all registers to be used (AX, BX, CX, DX, SI) onto the stack. After loading the LAP_COUNT value into the CX register, it performs exception handling via the jcxz .ldone instruction to immediately terminate the output without entering the loop if the number of stored lap times is 0. If a lap time exists, it initializes BX, the loop counter, to 0 and enters the .l_loop region. To perform an independent output for each row, it temporarily evacuates the BX value, which is the current loop sequence number, onto the stack via push bx. At this time, since the stack pointer moves downward, the current loop index can be directly referenced through the [esp] address. To control the layout so that it is rendered one line below each time a lap time accumulates, it retrieves the current loop index value ([esp]) into the row coordinate DH, adds the starting offset of 8 (add dh, 8), fixes the column coordinate DL to 4, and calls gotoxy. It first renders the current lap number (the value obtained by adding 1 to the index) and the pipe symbol (|) format on the screen by combining print_string and print_two_digit. Subsequently, to retrieve the actual stored value from the array, it retrieves the current loop index value ([esp]) backed up on the stack again, performs a left shift (shl ax, 1) operation to convert it into a 2-byte offset address, and transfers it to the SI register. Based on that address, it sequentially reads the integer data from [LAP_HOUR + si], [LAP_MIN + si], and [LAP_SEC + si], and completes the values line-by-line in the HH:MM:SS format through print_two_digit and print_char. When the output for one line is finished, it restores the

evacuated BX (pop bx), increments it by 1 (inc bx), decrements the CX value by 1 through the loop .l_loop instruction, and repeats the loop until all lap time data is outputted and then terminates.

```
delay:
    push ax
    push bx
    push cx
    push dx

.read_current:
    mov ah, 02h
    int 1Ah
    jc .read_current
    mov bl, dh

.wait_change:
    mov ah, 02h
    int 1Ah
    jc .wait_change
    cmp dh, bl
    je .wait_change

.done:
    pop dx
    pop cx
    pop bx
    pop ax
    ret

clear_screen:
    mov ax, 0600h
    mov bh, 17h
    mov cx, 0000h
    mov dx, 174Fh
    int 10h
    mov ax, 0600h
    mov bh, 70h
    mov cx, 1800h
    mov dx, 184Fh
    int 10h
    ret

%assign total_image_size 16384
times total_image_size - ($ - $$) db 0
```

The delay label is a function that generates an exact 1-second delay throughout the program by utilizing the Real-Time Clock (RTC) hardware interrupt. To prevent corruption of the existing register values currently in use during the execution of the function, it backs up AX, BX, CX, and DX onto the stack.

The read_current label invokes the INT 1Ah, AH=02h (Read Current Time from RTC) interrupt. If the read operation fails due to the hardware status, the Carry Flag is set (CF=1), so it continuously reads the current time again until successful via the jc .read_current instruction. Upon success, the 'current second' value contained in the DH register is temporarily stored in the BL register.

The wait_change label invokes the interrupt again to continuously read the current second value of the RTC in real-time. Afterward, through the cmp dh, bl instruction, it compares the just-retrieved second (DH) with the previously stored second (BL). If the two values match, it means 1 second has not yet passed, so it loops and waits infinitely due to the je .wait_change condition. The moment the second value of the hardware RTC chipset changes, it escapes the loop, and through this method, it resolves the CPU clock dependency problem of simple arithmetic loops (loop $) and achieves an accurate hardware-based 1-second synchronization. Finally, it restores the backed-up registers and returns via ret.

The clear_screen label is a split initialization function that cleanly erases the entire screen by designating different background colors for the upper region (main screen) and the bottommost region (status bar).

Upper Region Initialization: It configures AX=0600h (BIOS Video Services - Scroll Up Window and Clear Entire Screen) and designates the background and text color attribute as BH=17h (light gray text on a blue background). By setting the starting top-left coordinates to CX=0000h (Row 0 Column 0) and the ending bottom-right coordinates to DX=174Fh (Row 23 Column 79), it invokes the interrupt (INT 10h) to clear the main region in blue.

Bottom Status Bar Initialization: In the same manner, it configures AX=0600h, but inverts the color attribute to BH=70h (black text on a light gray background). By setting the starting coordinates to CX=1800h (Row 24 Column 0) and the ending coordinates to DX=184Fh (Row 24 Column 79), it independently initializes only the bottommost single line as a gray bar region for guidelines and then returns via ret.

The %assign total_image_size 16384 and times macro region are low-level padding specifications to accurately match the final binary file size of the written program to

exactly 16KB (16,384 bytes). After calculating the physical size of the actual code written up to the current point ($ - $$), it fills the remaining space deficient from the configured total image size with 0 (db 0), which is meaningless data, thereby forcibly satisfying the disk sector and virtual machine load specifications of IBM PC compatible systems.

5. Complete Source Code

```
[BITS 16]
ORG 0x7C00

; BOOT SECTOR
boot_start:
    xor ax, ax
    mov ds, ax
    mov es, ax
    mov ss, ax
    mov sp, 0x7C00

    mov ah, 02h
    mov al, 15
    mov ch, 0
    mov cl, 2
    mov dh, 0
    mov bx, 0x7E00
    int 13h
    jc boot_start

    jmp 0x0000:0x7E00

times 510-($-$$) db 0
dw 0xAA55


clock_program_start:
    jmp main_menu

; DATA

S_HOUR          dw 0
S_MINUTE        dw 0
S_SECOND        dw 0
T_HOUR        dw 0
T_MINUTE      dw 0
T_SECOND      dw 0
LAP_COUNT     dw 0
LAP_HOUR      times 10 dw 0
LAP_MIN       times 10 dw 0
LAP_SEC       times 10 dw 0

menu_title        db 'All-rounder Clock Setup', 0
title_underbar   db 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205, 205,
205, 205, 205, 0
menu_line1        db 'Welcome to All-rounder Clock.', 0
menu_line2        db 'This Setup prepares All-rounder Clock to run on your computer.', 0
menu_line3        db 249, ' To use Stopwatch now, press 1.', 0
menu_line4        db 249, ' To use Timer now, press 2.', 0
menu_key           db '1=Stopwatch    2=Timer', 0
stopwatch_text   db 'Stopwatch                 ', 0
stopwatch_key     db 'P=Pause    L=Lap    R=Reset    F3=Quit', 0
stopwatch_pause db 'S=Start', 0
timer_text        db 'Timer                          ', 0
timer_key          db 'P=Pause    F3=Quit', 0
timer_pause       db 'S=Start', 0
timeout_key       db 'F3=Exit               ', 0
input_hour_msg   db 'Enter Hour    (00-23): ', 0
input_min_msg    db 'Enter Minute (00-59): ', 0
input_sec_msg    db 'Enter Second (00-59): ', 0
timeout_msg       db 'Times Out!', 0
```

```
; MAIN MENU
main_menu:
    call clear_screen
    mov si, menu_title
    call draw_header
    mov dh, 4
    mov dl, 4
    call gotoxy
    mov si, menu_line1
    call print_string
    mov bl, 17h
    mov dh, 6
    mov dl, 4
    call gotoxy
    mov si, menu_line2
    call print_string
    mov dh, 8
    mov dl, 8
    call gotoxy
    mov si, menu_line3
    call print_string
    mov dh, 10
    mov dl, 8
    call gotoxy
    mov si, menu_line4
    call print_string
    mov si, menu_key
    call draw_statusbar

wait_menu:
    mov ax, 0100h
    int 16h
    jz wait_menu
    mov ah, 00h
    int 16h
    cmp al, '1'
    je sw_main
    cmp al, '2'
    je timer_main
    jmp wait_menu

go_main:
    jmp main_menu

; STOPWATCH

sw_main:
    call clear_screen
    mov word [S_HOUR], 0
    mov word [S_MINUTE], 0
    mov word [S_SECOND], 0
    mov word [LAP_COUNT], 0
    mov si, stopwatch_text
    call draw_header
    mov si, stopwatch_key
    call draw_statusbar

sw_display:
    mov bl, 17h
    mov dh, 5
    mov dl, 4
    call gotoxy
    mov ax, [S_HOUR]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [S_MINUTE]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [S_SECOND]
    call print_two_digit

    call delay
    mov ah, 01h
    int 16h
    jz sw_inc
    mov ah, 00h
    int 16h
    or al, 20h
    cmp al, 'p'
    je sw_pause
    cmp al, 'l'
    je sw_lap
    cmp al, 'r'
    je sw_main
    cmp ah, 3Dh
    je go_main
    jmp sw_inc

sw_pause:
    mov si, stopwatch_pause
    call draw_statusbar

sw_wait:
    mov ah, 01h
    int 16h
    jz sw_wait
    mov ah, 00h
    int 16h
    or al, 20h
    cmp al, 's'
    je sw_resume
    cmp al, 'l'
    je sw_lap_paused
    cmp al, 'r'
    je sw_main
    cmp ah, 3Dh
    je go_main
    jmp sw_wait

sw_resume:
    mov si, stopwatch_key
    call draw_statusbar
    jmp sw_display

sw_lap:
    call do_lap
    jmp sw_inc

sw_lap_paused:
    call do_lap
    jmp sw_wait

    add word [S_SECOND], 1
    cmp word [S_SECOND], 60
    jl sw_display
    mov word [S_SECOND], 0
    add word [S_MINUTE], 1
    cmp word [S_MINUTE], 60
    jl sw_display
    mov word [S_MINUTE], 0
    add word [S_HOUR], 1
    cmp word [S_HOUR], 24
    jl sw_display
    mov word [S_HOUR], 0
    jmp sw_display

; TIMER

timer_main:
    call clear_screen
    mov si, timer_text
    call draw_header
    mov dh, 5
    mov dl, 4
    call gotoxy
    mov si, input_hour_msg
    call print_string
    call bios_input_two_digit
    mov [T_HOUR], ax
    mov dh, 7
    mov dl, 4
    call gotoxy
    mov si, input_min_msg
    call print_string
    call bios_input_two_digit
    mov [T_MINUTE], ax
    mov dh, 9
    mov dl, 4
    call gotoxy
    mov si, input_sec_msg
    call print_string
    call bios_input_two_digit
    mov [T_SECOND], ax
    call clear_screen
    mov si, timer_text
    call draw_header
    mov si, timer_key
    call draw_statusbar

timer_display:
    mov bl, 17h
    mov dh, 5
    mov dl, 4
    call gotoxy
    mov ax, [T_HOUR]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [T_MINUTE]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [T_SECOND]
    call print_two_digit

    mov ax, [T_HOUR]
    or  ax, [T_MINUTE]
    or  ax, [T_SECOND]
    jz  timer_timeout

    call delay
    mov ah, 01h
    int 16h
    jz timer_dec
    mov ah, 00h
```

```
timer_pause_fn:
    mov si, timer_pause
    call draw_statusbar
timer_wait_resume:
    mov ah, 01h
    int 16h
    jz timer_wait_resume
    mov ah, 00h
    int 16h
    or al, 20h
    cmp al, 's'
    je timer_resume
    cmp ah, 3Dh
    je go_main
    jmp timer_wait_resume
timer_resume:
    mov si, timer_key
    call draw_statusbar
    jmp timer_display

timer_dec:
    cmp word [T_SECOND], 0
    jg dec_sec
    cmp word [T_MINUTE], 0
    jg borrow_min
    cmp word [T_HOUR], 0
    je timer_display
    dec word [T_HOUR]
    mov word [T_MINUTE], 59
    mov word [T_SECOND], 59
    jmp timer_display
borrow_min:
    dec word [T_MINUTE]
    mov word [T_SECOND], 59
    jmp timer_display
dec_sec:
    dec word [T_SECOND]
    jmp timer_display

timer_timeout:
    call clear_screen
    mov bl, 17h
    mov dh, 12
    mov dl, 35
    call gotoxy
    mov si, timeout_msg
    call print_string
    mov si, timeout_key
    call draw_statusbar
wait_timeout:
    mov ah, 01h
    int 16h
    jz wait_timeout
    mov ah, 00h
    int 16h
    cmp ah, 3Dh
    je go_main
    jmp wait_timeout

; PROCEDURES
gotoxy:
    push ax
    push bx
    mov ah, 02h
    mov bh, 0
    int 10h
    pop bx
    pop ax
    ret

draw_statusbar:
    mov bl, 70h
    mov dh, 24
    mov dl, 4
    call gotoxy
    call print_string
    ret

print_string:
    push ax
    push si
.print_loop:
    lodsb
    or al, al
    jz .print_done
    call print_char
    jmp .print_loop
.print_done:
    pop si
    pop ax
    ret

print_char:
    push ax
    push bx
    mov ah, 0Eh
    mov bh, 0
    int 10h
    pop bx
    pop ax
    ret

print_two_digit:
    push ax
    push bx
    push cx
    mov cl, bl
    xor ah, ah
    mov bl, 10
    div bl
    mov bl, cl
    add al, '0'
    call print_char
    mov al, ah
    add al, '0'
    call print_char
    pop cx
    pop bx
    pop ax
    ret
bios_input_two_digit:
    push bx
    push cx
    mov bl, 17h
.w1:
    mov ah, 00h
    int 16h
    cmp al, 13
    je .sd
    cmp al, '0'
    jl .w1
    cmp al, '9'
    jg .w1
    call print_char
    sub al, '0'
    mov cl, al

do_lap:
    mov ax, [LAP_COUNT]
    cmp ax, 10
    jge .done
    mov bx, ax
    shl bx, 1
    mov ax, [S_HOUR]
    mov [LAP_HOUR + bx], ax
    mov ax, [S_MINUTE]
    mov [LAP_MIN + bx], ax
    mov ax, [S_SECOND]
    mov [LAP_SEC + bx], ax
    inc word [LAP_COUNT]
    call print_laps
.done:
    ret

print_laps:
    push ax
    push bx
    push cx
    push dx
    push si
    mov cx, [LAP_COUNT]
    jcxz .ldone
    mov bx, 0
.l_loop:
    push bx
    mov bl, 17h
    mov dh, [esp]
    add dh, 8
    mov dl, 4
    call gotoxy
    mov si, .lap_str
    call print_string
    mov ax, [esp]
    inc ax
    call print_two_digit
    mov al, ' '
    call print_char
    mov al, '|'
    call print_char
    mov al, ' '
    call print_char
    mov ax, [esp]
    shl ax, 1
    mov si, ax
    mov ax, [LAP_HOUR + si]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [LAP_MIN + si]
    call print_two_digit
    mov al, ':'
    call print_char
    mov ax, [LAP_SEC + si]
    call print_two_digit
    pop bx
    inc bx
    loop .l_loop
.ldone:
    pop si
    pop dx
    pop cx
    pop bx
    pop ax
    ret
.lap_str db 'Lap ', 0
```

```
delay:
    push ax
    push bx
    push cx
    push dx

.read_current:
    mov ah, 02h
    int 1Ah
    jc .read_current
    mov bl, dh

.wait_change:
    mov ah, 02h
    int 1Ah
    jc .wait_change
    cmp dh, bl
    je .wait_change

.done:
    pop dx
    pop cx
    pop bx
    pop ax
    ret

clear_screen:
    mov ax, 0600h
    mov bh, 17h
    mov cx, 0000h
    mov dx, 174Fh
    int 10h
    mov ax, 0600h
    mov bh, 70h
    mov cx, 1800h
    mov dx, 184Fh
    int 10h
    ret

%assign total_image_size 16384
times total_image_size - ($ - $$) db 0
```

6. Findings & Benefits

6.1. Technical Findings

- **Control of Register Pollution:** BIOS interrupts and specific arithmetic operations like division (div) arbitrarily modify the values of existing registers (such as AX and BX). Through this project, a deep, low-level understanding of memory management tactics was acquired, specifically regarding how to safely evacuate and restore volatile data using the stack (push/pop) or spare registers (CL) to consistently maintain text color attributes on the screen.

- **Mechanism of Text-Mode Video Attributes:** A critical hardware characteristic was discovered and successfully integrated into the codebase: when clearing the screen via INT 10h, AH=06h, the attribute byte loaded into the BH register determines not only the background color but also definitively dictates the default font color of the coordinates where the cursor will subsequently pass.

6.2. Benefits & Conclusion

- **Maximized Independence and Lightweight Design:** Complete optimization was achieved by constructing a fully functional clock hardware control system utilizing less than 16KB of pure machine code, completely independent of any massive operating system kernel infrastructure.

- **Highly Modularized Architecture:** A highly reliable and robust architecture was accomplished by fully decoupling and separating the core operational procedures—screen layout control, string rendering, numerical output, and asynchronous input polling loops—thereby maximizing code maintainability, scalability, and reusability.

- **Extended Functionality beyond Basic Clock Systems:** The system was extended to incorporate a dedicated Stopwatch mode with real-time Lap-Time logging (up to 10 records) and a user-configurable countdown Timer with timeout notification — features that collectively elevate the project from a basic timekeeping utility to a comprehensive, multi-functional bare-metal clock suite.

7. References

- 8086 BIOS and DOS Interrupts Reference (https://yassinebridi.github.io/asm-docs/8086_bios_and_dos_interrupts.html)

- Complete 8086 Instruction Set Reference (https://www.eng.auburn.edu/~sylee/ee2220/8086_instruction_set.html)

- 8086 Assembly Projects Reference (https://github.com/yousefkotp/8086-Assembly-Projects/tree/main)

- Digital Clock Reference https://github.com/etkey/Digital-Clock-in-Assembly-8086